

IGR & REAL ESTATE DATA PROCESSING PIPELINE

Comprehensive Technical Architecture & Step-by-Step System Documentation (project.py)

Automated Ingestion, ML Clustering, Multi-File Merging, Spatial Enrichment, 20-Step Pipeline, Web Dashboard & PostgreSQL Loading

1. Executive Summary & Core Mission

project.py serves as the master production orchestrator responsible for transforming raw, semi-structured Inspector General of Registration (IGR) filings and international real estate transaction datasets into a standardized, validated, and enriched relational format. Operating across domestic micro-markets (Mumbai, Pune, Thane) and international markets (Dubai, Abu Dhabi), the pipeline automates nested JSON parsing, machine-learning phonetic/n-gram project clustering, Marathi land area conversion (Hectare-Are-Guntha to Sq.m), automated multi-file human review merging with missing-column matrix audit reporting, MahaRERA master reconciliation, automated Non-RERA (NR) sequencing, buyer demographics geo-enrichment, automated quality-checker notification emails, columnar Parquet export, PostgreSQL upload, and statistical outlier classification. Shared core logic is maintained in **pipeline_core.py**, supporting both interactive terminal execution and the real-time SSE Web Dashboard (**server.py**).

2. Pipeline Execution Modes

Mode	Scope & Steps	Operational Workflow & Use Case
Mode [1]	Run from START Steps 1 to 20 (Full End-to-End)	Processes freshly delivered LLM files from Google Drive (2. <i>LLM Processed Data</i>). Runs entity clustering, outputs manual review file, pauses for operator verification, resumes upon confirmation, aligns schema, converts to Parquet, uploads to DB, and flags outliers.
Mode [2]	Run from STEP 7 Steps 7 to 20 (Resume Mode)	Directly loads verified/corrected manual review workbooks from Google Drive (3. <i>Manually Corrected</i>). Supports single, multiple, or all files with automated schema harmonization and missing-column audit reporting. Executes RERA matching, NR index assignment, coordinate enrichment, Parquet export, DB upload, and outlier classification.
Mode [3]	Run STEP 18 ONLY Steps 18 to 20 (Direct Parquet)	Fast-path for converting verified final Excel/CSV datasets directly to compressed Apache Parquet format. Automatically proceeds to database ingestion via <code>final_code.py</code> (Step 19) and statistical outlier detection (Step 20).
Mode [4]	Run STEP 20 ONLY Step 20 Only (Outlier Update)	Standalone statistical outlier detection and classification directly on PostgreSQL database records. Computes P1/P99 trimmed medians, updates flags (<code>rate</code> , <code>is_outlier</code> , <code>outlier_type</code>), and exports multi-sheet audit workbooks without re-running data ingestion.
Dubai Mode	2-Step Workflow Scraping & Processing (Dubai DLD)	Dedicated workflow for Dubai Land Department (City ID: 15). Step 1 executes automated Selenium scraper (<code>dubai_scraper.py</code>) with automated date sync from DB. Step 2 executes end-to-end processing pipeline (<code>dld_pipeline.py</code>) with coordinate enrichment and NR indexing.

3. Detailed Step-by-Step Technical Guide (Steps 1 to 20)

Pre-Step: City Configuration & Google Drive Binding

Initializes the active city context via `city_config.py` (Pune: 9, Mumbai: 8, Thane: 12, Dubai: 15, Abu Dhabi: 1). Dynamically binds to Google Drive desktop shortcuts on `G:\shortcut-targets-by-id\...` for input, review, and final folders. Loads city carpet area divisors (Mumbai: 1.45, Pune: 1.35, Thane: 1.40, Dubai: 1.0 from `divisor.py`).

Step 1: Input File Discovery (Location-Wise Drive Browser)

Recursively scans 2. *LLM Processed Data* on Google Drive. Groups and displays available localities (e.g., Andheri, Kothrud, Wakad). Captures selected location globally for structured subfolder export downstream.

Step 2: DictToColumn JSON & Dictionary Extraction

Executes `DictToColumn.py`. Flattens nested stringified JSON dictionary representations from raw IGR metadata columns into typed `DataFrame` columns. Automatically verifies target city identity.

Step 3: Standardization of Initial Project Name

Populates final_project_name from project_name_en. Falls back to building_name_en if blank, recording audit provenance in final_project_name_status ('Project Name Considered' vs 'Building Name Considered').

Step 4: Static Dictionary Transaction Type Mapping

Imports static.py. Translates localized Marathi deed classifications (Kharedikhat, Bhadepatta, Bakshispatra, etc.) into standardized English transaction types (Sale Deed, Lease Deed, Gift Deed, Mortgage Deed). Reports value counts of 100% matched transaction types.

Step 5: High-Level Transaction Categorisation

Applies transaction_categorizer.py to categorize transactions into primary analytical buckets: **Sale** (capital purchases/conveyances), **Lease / Mortgage** (rental agreements and bank liens), and **Other**.

Step 5.5: Marathi Revenue Village to Locality Mapping

Queries PostgreSQL public.transactions to map village_name_marathi / areaname to canonical location_name and registered_document_village_name, ensuring zero data loss for municipal revenue villages.

Step 6: Hybrid Project Clustering & Marathi Area Conversion (_for_manual.xlsx)

Core ML component via project_name_Std_and_area_conversion.py:

- **Part A (Entity Resolution)**: Strips legal suffixes (CHS, Phase, Wing); builds character n-gram TF-IDF matrices; computes pairwise cosine nearest neighbours; applies conservative rejection heuristics; executes greedy clustering.
- **Part B (Marathi Area Parsing)**: Regex-parses Marathi units (Hectare-Are-Sq.m, Guntha, Are, Sq.ft, Sq.m) into normalized square metres.
- **Part C (Unit & Floor)**: Extracts flat numbers (flat_number) and floor numbers (floor_number) via regex and floor mapping.
- **Part D (Carpet & Rate Derivation)**: Applies city divisors to derive net_carpet_area_sq_m. Calculates rate_in_sqft and creates is_manual_processed flag ('Yes' for Sale >= 70,000 INR).
- **Clean Export**: Drops internal helper columns and exports multi-sheet workbook <Location>_for_manual.xlsx to Google Drive (3. *Manually Corrected*/<Location>).

Step 7: Multi-File Ingestion, Schema Audit & Merging Engine

Implemented in pipeline_core.py (load_and_merge_step7_files):

- Supports loading a single file, multiple comma-separated files, or 'all' location files.
- Pre-maps raw columns: flat_no → flat_no_raw, floor_no → floor_no_raw.
- Evaluates each file against the union of all columns, detecting missing fields per file.
- Automatically saves merged dataset to Google Drive: 3. *Manually Corrected* / FINAL MERGE / final merge file.xlsx.
- Generates 3-sheet audit report Step7_Missing_Columns_Report.xlsx (Summary, Column Matrix, Detail) and CSV matrix.
- Strips over 100 internal and unneeded helper columns (e.g., society_name_en, FLAT AREAS, AREA DETAILS_*).

Step 8: Standardize Database Column Schema

Executes rename_columns() to align raw fields to production PostgreSQL schema: docno → document_number, consideration_amt → agreement_price, marketvalue → guideline_value, sellerparty → seller_name, purchaserparty → buyer_name, flat_number → unit_number, srocode → sub_registrar_office_code.

Step 9: Categorise Property Types

Normalizes property descriptions into six canonical types: **Flat** (Apartments, Rooms, Houses), **Office** (Commercial Offices), **Shop** (Retail, Restaurants), **Villa** (Bungalows, Row Houses), **Plot** (Open Land), and **Others**.

Step 10: Buyer Location & Postal PIN Code Enrichment

Uses regex to extract 6-digit Indian PIN codes from buyer_name. Merges against postal_pincode.csv to append buyer_locality, buyer_district, and buyer_state.

Step 11: MahaRERA Master Dataset Matching & BHK Derivation

Runs rera_matching.py against city-specific MahaRERA grand files. Handles nested JSON sold unit metrics and carpet range fallbacks to append verified project names, geocoded coordinates, official RERA index, and unit configuration (BHK). Non-RERA cities safely bypass with null placeholders.

Intermediate: Financial & Chronological Calculations

- Derives registration quarters (e.g. Q1-2026) and standardizes date formatting to DD/MM/YYYY.
- Computes net carpet area in square feet: `net_carpet_area_sqft = net_carpet_area_sq_m * 10.7639`.
- Computes calculated rate: `rate = agreement_price / net_carpet_area_sqft` (standardized to column rate).
- Assigns DB defaults: `state_name, country_name, data_source='lgr', is_llm_processed='Yes'`.

Step 12: Sequential Non-RERA (NR) Index Assignment

Transactions without a MahaRERA match receive an incremental Non-RERA ID. Queries PostgreSQL for current max sequence (`SELECT MAX(internal_index_id) WHERE internal_index_id ~* '^nr\d+'`) and assigns new sequential IDs (nr18421, nr18422, etc.).

Step 13: Fetch Location Coordinates from Database

Queries `public.dim_location` for `location_latitude` and `location_longitude` based on `location_name`, providing locality-level geospatial fallbacks for every record.

Step 14: Project Coordinates via Google Places API (Optional)

Toggleable geocoding module (`project_coordinates.py`) to query high-precision project coordinates for unmapped developments using Google Places API.

Step 15: Filter & Order Selective Database Columns

Imports `DB_SEQUENCE` from `db_columns.py`. Strictly reorders and filters DataFrame columns to mirror the exact positional schema of `public.transactions` in PostgreSQL.

Step 16: Title Case Normalization

Applies Python `.title()` formatting across all object/string columns for cosmetic and visual uniformity.

Step 17: Final Dataset Export & Automated Checker Email Notification

- Saves master cleaned workbook `<Location>_final_processed.xlsx` to Google Drive: *4. Final processed file/<Location>/.*
- **Automated Email Dispatch:** Launches background thread via `send_final_checker_email()` in `city_config.py`. Dispatches an automated HTML alert to the quality checker (`deeksha@sigmavalue.co.in`) with city, locality, row count, file path, and Google Drive share link.

Step 18: High-Performance Parquet Conversion

Executes `parquet_conversion.py` to convert the final dataset into an optimized Apache Parquet columnar file under `converted_feather_parquet/<City>/<City>_db1.parquet` with date string normalization (safe from 64-bit nanosecond overflows).

Step 19: Trigger Central Database Upload Pipeline

Executes `DB1_DB2_Uploading_Pipeline/final_code.py` via subprocess in the active Python virtual environment to ingest the Parquet records directly into PostgreSQL tables.

Step 20: Post-Upload Statistical Outlier Detection (`outlier_update.py`)

Statistical outlier classification engine (`outlier_update.py`):

- **Grouping & Trimming:** Groups by (city, location, category, property_type) (international markets collapsed into `__ALL_LOCATIONS__`). Trims 1st and 99th percentiles (P1/P99) on `calc_rate`.
- **Trimmed Median Bounds:** Sets `lower_limit = median / 4` and `upper_limit = median * 4`.
- **Classifications:** Flags records as Normal, Lower Outlier, Upper Outlier, Mumbai Low Price Outlier (Sale < 1L), Mumbai Zero Area & Rate - Not Outlier, or Invalid/Excluded.
- **Database Update:** Performs batch update on PostgreSQL columns: `rate, is_outlier, and outlier_type` via `execute_values`.

4. Web Dashboard Architecture (`server.py` & `web/`)

A full-stack, browser-based management interface is integrated in `Processing/server.py` and `Processing/web/`. Built with a high-performance HTTP/FastAPI backend and modern CSS/JS frontend, it provides:

- **Real-Time SSE Streaming:** Server-Sent Events stream live terminal logs from the pipeline directly to the browser UI with auto-scroll and status pills.
- **Multi-File Review Selector:** Allows operators to pick multiple review files directly from the browser UI with instant merge audit feedback.
- **Dynamic Database Switcher:** Dynamically change the active PostgreSQL database via `/api/set-db` on the fly.
- **Dynamic Mode Routing & Auto-Scroll:** Form cards automatically expand and focus for Mode 1 (Drive Picker), Mode 2 (Step 7 Resume), Mode 3 (Parquet), and Mode 4 (Outliers).
- **Interactive Drive Explorer:** Recursively scans local Google Drive shortcut trees for input, manual review, and final file locations.

5. International Real Estate Processing (Dubai & Abu Dhabi)

The system natively accommodates international datasets from the Middle East:

- **Automated Portal Scraper (Processing/Dubai_processing/dubai_scraper.py):** Automated Selenium scraper for the Dubai Land Department (DLD) Open Data portal. Queries DB for the latest transaction date to fetch incremental data, handles reCAPTCHA, and uploads to Google Drive.
- **End-to-End Processing (dld_pipeline.py):** Normalizes DLD transaction records, enriches coordinates from PostgreSQL, allocates sequential NR indexes (nrXXXX), and produces database-ready outputs.
- **Unified Outlier Grouping:** For Dubai (City ID: 15) and Abu Dhabi (City ID: 1), all micro-locations are collapsed into `__ALL_LOCATIONS__` in `outlier_update.py` to compute robust emirate-wide medians.
- **Historical Calendar Serialization:** Date parsing preserves historical and Hijri dates as safe strings without conversion failures.

6. Master File & Dependency Registry

File / Module	Role & Core Responsibility
project.py	Master CLI orchestrator coordinating full 20-step execution pipeline.
pipeline_core.py	Shared core library: DB parameters, Step 7 multi-file merging, schema renaming, NR indexing, village mapping.
server.py	FastAPI/HTTP Web Server providing real-time SSE progress streaming, dynamic DB switcher, and web dashboard.
web/ (index.html, app.js, style.css)	Modern browser operations interface with mode routing, live log viewer, and Drive explorer.
city_config.py	Central registry for city IDs, Google Drive URLs/IDs, RERA paths, and automated checker email dispatcher.
divisor.py	Carpet area divisors by city (Mumbai: 1.45, Pune: 1.35, Thane: 1.40, Dubai: 1.0).
DictToColumn.py	JSON flattener extracting stringified dictionary entries into typed DataFrame columns.
transaction_categorizer.py	Rule-based transaction categorizer classifying deeds into Sale, Lease/Mortgage, and Other.
project_name_Std_and_area_conversion.py	TF-IDF n-gram clustering, Marathi area converter, unit/floor parser, and manual review workbook exporter.
rera_matching.py	Fuzzy matching engine reconciling project names, RERA indexes, coordinates, and BHK configurations against MahaRERA master records.
postal_pincode.csv	Master Indian postal directory for buyer locality, district, and state geo-enrichment.
db_columns.py	Canonical PostgreSQL column sequence definition (DB_SEQUENCE).
parquet_conversion.py	High-speed columnar Parquet conversion utility with type validation and date string normalization.
outlier_update.py	Statistical outlier detection engine calculating trimmed medians and updating PostgreSQL flags via <code>execute_values</code> .
Dubai_processing/dubai_scraper.py	Automated Selenium scraper and Drive uploader for Dubai Land Department real estate transaction records.
Dubai_processing/dld_pipeline.py	End-to-end data transformation, coordinate enrichment, and NR index allocator for Dubai DLD records.
generate_pdf_doc.py	ReportLab script that compiles this comprehensive documentation into a publication-quality PDF.